

大数据运维面试速查手册

面试：2026-09-27 | [重点]重点，[必考]必考 | 按配套 PDF 固定为 5 页。

本手册用短问短答满足 5 页限制；200-300 字完整口语稿见三个案例及《面试问答脚本》。学习基线：Hadoop 3.3.6、Spark 3.5.x，Hive 示例按 3.1.x + Tez；公司实际版本待确认。Linux 命令供理解，不需要今晚搭环境。

第 1 页 | [必考] HDFS：谁管理，谁存储

考点	一句话理解	高频问法与直接回答
NameNode / DataNode	NN 记文件目录、块位置等元数据；DN 存真实数据块，定期报心跳和块清单。	问：文件内容经过 NN 吗？答：一般不经过，客户端向 NN 查位置，再直接和 DN 传数据。
Secondary NameNode	定期把 fsimage（元数据快照）与 edits（修改记录）合并成检查点，帮助控制恢复成本。	问：NN 挂了它接班吗？答：不能。它不是热备；HA 的 Standby NN 才有接管能力。
块大小	Hadoop 2/3 常见默认 128 MiB，即 134217728 字节，可配置；老版本曾默认 64 MiB。	问：为什么 128M？答：大块能减少寻址和元数据开销，又保留一定并行度；这是常见折中，不是所有业务最优。1 KiB 小文件不会硬占满 128 MiB。
副本和机架	普通副本文件默认 3 份；典型放法是第一份靠近写入端，第二份跨机架，第三份与第二份同机架不同机器。	问：为何不全放一台？答：一台坏了就全没了；跨机架还能抵抗机架故障。3 副本不等于必须放 3 个机架，也不是备份。
HA 高可用	Active 工作，Standby 跟进共享编辑日志；ZooKeeper 做协调，ZKFC 监测健康并参与选主，切换要隔离旧主。	问：ZooKeeper 直接复制文件吗？答：不。典型 HA 用 JournalNode 保存共享 edits，文件块仍在 DN；隔离旧主是防止两个主同时写。

[重点] 读写流程，用这张图讲：

写：Client -> NN（权限检查、申请块和 DN 列表）
Client -> DN1 -> DN2 -> DN3（数据包管道）
Client <- DN1 <- DN2 <- DN3（逐级确认）-> 完成文件
读：Client -> NN（问块位置）-> 最近可用 DN（读块、校验）
DN 不可用或校验失败 -> 换另一个副本

问：写入为什么不让 NN 转发？答：NN 专心管目录和位置，数据分散到 DN 直接传，避免所有大文件都挤过一个管理节点。传输和确认按数据包流水进行，不是整块写完才继续传。

问：安全模式是什么？答：是一种限制修改的保护状态。启动时会等块上报达到条件；NN 本地资源不足也可能触发。先查原因，不能见到报错就强退。DN 盘满本身不等于自动进入安全模式。

来源：HDFS 架构、用户指南、HA、默认配置。

第 2 页 | [必考] YARN：把资源分给应用

角色	大白话	高频问法与直接回答
ResourceManager (RM)	集群总管，负责接收应用和调度资源。	问：RM 替 Spark 做计算吗？答：主要分资源；应用自己的 AM 协调执行。
NodeManager (NM)	每台机器上的管理员，启动和监控 Container，汇报节点情况。	问：节点坏了看谁？答：看 NM 心跳、节点健康、机器资源，再看 RM 的节点列表。
ApplicationMaster (AM)	每个应用的协调者，向 RM 申请资源，并和 NM 配合启动运行。	问：全局只有一个 AM 吗？答：不是，每个应用的运行尝试有自己的 AM。
Container	一份内存、CPU 等资源及其运行环境。	问：它就是 Docker 吗？答：不是，这里是 YARN 的资源分配与执行单位。

[必考] 问：YARN 的任务提交流程是什么？

Client 提交应用与启动材料 -> RM 接收
RM 分配首个 Container -> 某台 NM 启动 AM
AM 向 RM 注册并申请资源 -> RM 分配后由 NM 启动计算进程
应用执行、汇报状态 -> 完成后 AM 注销，释放资源

答：先提交，再拿资源启动 AM；AM 再申请执行资源并协调任务，完成后回收资源。Spark 的 cluster 模式 Driver 在集群侧 AM 进程中，client 模式 Driver 在提交端，不能混着讲。

调度器	怎么分	高频问法与直接回答
FIFO	主要按先后顺序分配。	问：缺点是什么？答：大任务容易让后来的短任务等很久。
Capacity	按队列预留容量，可设上限，符合条件时借用空闲资源。	问：有空闲还排队？答：还受队列上限、用户限制、AM 限额、节点标签等约束。
Fair	让活跃应用或池在一段时间里获得相对公平的资源份额，可带权重。	问：和 Capacity 有何区别？答：一个强调公平分享，一个强调队列容量保障，具体能力有重叠。

[重点] Capacity 适合多团队共享资源，但不能凭“国企”断定公司用了它。

问：Spark 一直 **ACCEPTED** 是什么情况？答：应用被接收，但尚未进入正常运行；常见是 AM 资源没分到。先看 RM 的 Diagnostics、队列容量及上限、AM 限额、节点可用资源，别直接归因于代码慢。

完整只读命令 / 入口	参与与用途
<code>yarn application -status application_171000000000_00_0042</code>	<code>application</code> 管理应用， <code>-status</code> 查状态与诊断；后面是示例 ID，换成实际 ID。
<code>yarn node -list -all</code>	<code>node</code> 查看节点， <code>-list</code> 列表， <code>-all</code> 包含所有状态。
<code>http://rm01:8088/cluster/scheduler</code>	典型 HTTP RM 调度页面； <code>rm01</code> 是示例主机。HTTPS、端口、代理和登录方式以部署为准。

来源：YARN 架构、Capacity、Fair、Spark on YARN。

第 3 页 | [必考] Spark：怎么计算，为什么会慢

考点	大白话	高频问法与直接回答
为什么通常更快	相比传统 Hadoop MapReduce 多轮计算，Spark 能缓存复用数据，并把多个步骤组织起来，减少中间落盘和启动成本。	问：因为全在内存吗？答：不是。Shuffle、溢写、输入输出都可能碰磁盘；一次简单扫描也不一定更快。
RDD	分散在多台机器上的逻辑数据集，按分区并行处理；变换产生新 RDD，可按依赖重算丢失分区。	问：数据都必须缓存吗？答：不，缓存需要选择；有些结果重新计算反而更划算。
DAG	有向无环的计算依赖图：先读、再过滤、再汇总。	问：为什么看 DAG？答：能找到慢在哪个阶段；遇到 Shuffle 依赖通常会划分 Stage。
Shuffle	为分组、关联等操作，把相同键的数据重新送到一起。	问：为什么耗性能？答：涉及网络传输、序列化、排序和磁盘读写；数据量大或分布不均时更明显。
数据倾斜	少数键特别热门，少数 Task 工作量远超其他 Task。	问：怎么确认？答：看同一 Stage 的任务耗时和输入 / Shuffle 数据量，少数任务显著偏大才有依据。
Driver Memory	Driver 用于计划、调度和部分结果管理的 JVM 堆。	问：给 Executor 加内存能救 Driver 吗？答：不能；把大量结果收回 Driver 可能撑爆 Driver，要限制收集量。
Executor Memory	Executor 跑 Task，堆内用于执行和缓存；容器还需要额外开销。	问：它等于 YARN 容器总内存吗？答：不。还可能包含 memoryOverhead、堆外及单独配置的 PySpark 内存。

[重点] 倾斜解决法：先确认热点，再选方法，最后核对结果。

小词典：Stage 是计算阶段；Task 通常处理一个分区；spill 是内存放不下时临时写盘；GC 是回收不再使用的内存。

方法	怎么理解	高频问法与直接回答
预聚合	能先在本地汇总的，就先汇总再搬运，减少传输量。	问：平均值怎么预聚合？答：先存总和与条数，最终再相除，不能直接平均各组平均值。
加盐	给热点键分多个小组，先分散处理，再去掉盐做正确的二次汇总。	问：随便加随机数行吗？答：不。聚合要还原，Join 还要配套复制或改写另一侧，确保不漏算、不重算。

还没启动 -> RM 队列 / AM 限额 / 节点资源
已经运行 -> Spark UI 找慢 Stage -> 看 Task 分布
-> 输入量 / Shuffle / spill / GC / 重试 -> 对症改动

问：内存不足就调大内存吗？答：先分清哪个进程报错，是否因为数据倾斜、一次收集太多、分区太大或缓存太多。资源要和队列余量一起看，盲目加大可能让应用更难拿到资源。

配置名速记：spark.driver.memory 是 Driver 堆；spark.executor.memory 是 Executor 堆；spark.executor.memoryOverhead 是 Executor 额外内存预算。它们不是 shell 命令，单位及生效方式看版本和部署模式。

来源：RDD 与 Shuffle、组件、性能优化、内存配置。

第 4 页 | [必考] Hive：用 SQL 描述分析

考点	大白话	高频问法与直接回答
Hive 本质	管理表的元数据，把 SQL 编译成执行计划，交给所配置的计算引擎；常见 Tez，历史版本也支持 MR / Spark。	问：Hive 本身是 HDFS 吗？答：不是，Hive 是 SQL 分析层；表数据可放在 HDFS，存储与计算分工不同。
分区表	按日期等条件分开组织数据。 <code>PARTITIONED BY</code> 声明分区列，过滤分区列可少读数据。	问：建分区就自动快吗？答：数据要落到正确分区、元数据要登记、条件要匹配；全历史查询仍可能读全部分区。
小文件	文件和块太多，增加 NN 元数据内存和查询文件打开、任务调度的开销。	问：怎么解决？答：从写入批次、分区粒度、输出文件数控制；存量按格式合并或重写，事务表走 compaction。
内部表	Hive 通常负责数据生命周期，删表一般连数据一起处理。	问：和外部表差别？答：外部表默认主要管理元数据，删表通常保留数据；还要查版本和 <code>purge</code> 属性，不能说绝不删数据。

[重点] 最小分区示例 (在已连接 Hive 的 SQL 会话执行；仅供学习)：

```
CREATE TABLE interview_events (  
  user_id STRING  
) PARTITIONED BY (dt STRING) STORED AS ORC;
```

```
EXPLAIN SELECT COUNT(*)  
FROM interview_events WHERE dt = '2026-09-25';
```

参数 / 关键字：`CREATE TABLE` 建表；`user_id STRING` 是用户标识字符串列；`PARTITIONED BY (dt STRING)` 声明日期分区列；`STORED AS ORC` 选择列式存储；`EXPLAIN` 只看计划；`COUNT(*)` 数行；`WHERE dt = ...` 限定一个日期分区。新表是空的，建表不会自动搬数据，完整迁移见案例三。

SQL -> 解析 / 查元数据 -> 优化与执行计划
-> 执行引擎申请资源并读数据 -> 汇总结果
按天分区 + 日期条件 -> 只读取需要的日期分区

[必考] 问：COUNT 跑了十分钟，一定全表扫描吗？ 答：不一定，先分清排队还是执行慢，再看执行计划和实际输入量。
`COUNT(*)` 有时能借助统计信息优化，不能看到 `COUNT` 就下结论；`COUNT(列)` 还会忽略该列的 `NULL`，`COUNT()` 不是这里的正确写法。

问：EXPLAIN 看什么？ 答：先看输入表、分区和路径，再看过滤条件、Join、聚合与数据交换。计划格式随版本和引擎变化；优化前后必须保持同一统计范围，并核对结果。

问：小文件一定用 CONCATENATE 吗？ 答：不是，它有格式和表类型限制。ORC / RCFile 等符合条件时才考虑；也不能把所有分区混成一个文件，或直接拼接底层数据文件。

来源：Hive DDL、EXPLAIN、内部与外部表、配置与执行引擎。

第 5 页 | [必考] Linux 排查 + [重点] 数据治理

以下在目标 Linux 机器执行，路径和主机名是示例；工具需已安装。网络和日志命令通常只读，读取部分进程信息需要相应权限。

完整命令	参数解释 / 看什么
<code>df -h /data; df -i /data</code>	<code>-h</code> 人类易读容量； <code>-i</code> 看 inode 数量。 <code>/data</code> 是目标挂载路径，盘没满也可能 inode 用光。
<code>du -sh /data/logs; free -h; top</code>	<code>du -s</code> 汇总目录， <code>-h</code> 易读单位； <code>free -h</code> 看内存，重点看 available； <code>top</code> 实时 CPU / 进程，按 q 退出。
<code>tail -n 100 -f /var/log/hadoop/hadoop.log</code>	<code>-n 100</code> 先看末尾 100 行， <code>-f</code> 持续跟随写入；Ctrl+C 退出。
<code>grep -n -E 'ERROR WARN' /var/log/hadoop/hadoop.log</code>	实际命令见下方代码， <code>-n</code> 行号， <code>-E</code> 扩展正则；竖线表示 “或”，表格里的转义不复制到终端。
<code>awk '{print \$1, \$2}' /var/log/hadoop/hadoop.log</code>	单引号保护程序；按空白切列，打印第 1、2 列，常用于提取时间等字段。
<code>ping -c 4 rm01; telnet rm01 8088</code>	<code>-c 4</code> 发 4 次 ICMP；telnet 尝试 TCP 端口，Ctrl+] 后输入 quit 退出；ping 通不代表端口通，禁 ICMP 也不代表宕机。
<code>netstat -lntp; ss -lntp</code>	两者均可查本机监听端口： <code>-l</code> 监听， <code>-n</code> 数字显示， <code>-t</code> TCP， <code>-p</code> 进程；ss 常作为新系统替代。
<code>hdfs dfsadmin -report</code>	dfsadmin 是 HDFS 管理入口， <code>-report</code> 查 DN 容量与状态；不能代替查看 NN 本地盘。

`grep -n -E 'ERROR|WARN' /var/log/hadoop/hadoop.log`

问：CPU / 内存 / 盘怎么查？ 答：先用 top、free、df 看整体，再用 du 定位目录；缓存占内存不等于内存耗尽，CPU 高也要看持续时间和对应进程。

问：日志怎么查？ 答：先对齐故障时间、应用 ID 和报错，再用 tail 看上下文、grep 定位关键行、awk 摘取字段，不能只拿最后一行异常猜原因。

问：端口不通怎么查？ 答：先确认主机解析和网络，再测试目标端口；到服务端看是否监听、绑定地址及防火墙，结合服务日志定位。

[必考] 问：HDFS 磁盘写满怎么办？ 答：先区分 NN 本地盘、DN 数据盘、inode 和 HDFS 目录配额。结合 df、du、report 及日志查来源；清理有保留依据的过期日志或扩容，保护元数据和块文件。安全模式恢复后才按流程处理 HDFS 过期数据，并复查读写。

治理考点	大白话 + 高频问答
元数据 / 血缘	元数据说明字段、分区、口径和负责人；血缘说明来源与去向。问：血缘怎么实现？答：采集 SQL、调度依赖与引擎事件，连成“源表→加工→目标表”的关系；Atlas Hive Hook 可自动采集部分关系，自定义脚本仍可能需要补录。
完整 / 一致 / 及时	看必填字段是否缺、上下游口径和数量能否对齐、是否按时到数。问：怎么落地质量检查？答：定空值率、重复数、对账差值、到数截止时间等规则，定时检查并告警；阈值和处理责任人跟业务确认。

来源：GNU coreutils、grep、gawk、Atlas Hive 血缘。核对日期：2026-09-25。